Week6_예습과제

CH5. 회귀(Regression)

5-1 회귀 소개

회귀 분석 : 데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향을 이용한 통계학 기법

회귀 in 자연 법칙 : 사람의 키는 평균 키로 회귀하려는 경향을 가짐.

회귀 :

여러 개의 독립변수 → 한개의 종속 변수 간 상관관계를 모델링하는 기법

$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2 + \cdots + \hat{\beta}_p X_p$$

Y : 종속 변수(결정값*/ 아파트 가격)

X1, X2, ... Xp : 독립 변수들(피처/ 방개수, 방크기, ...)

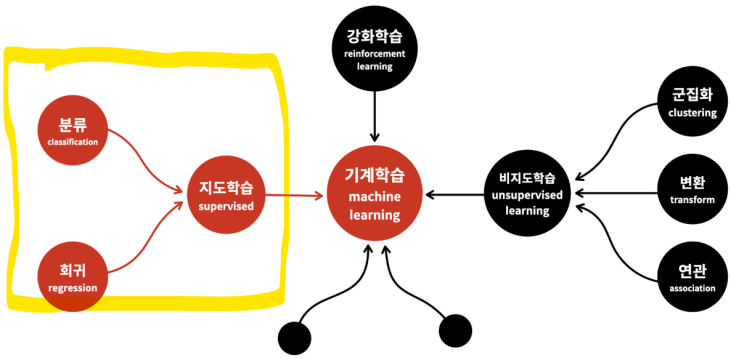
B0, B1, ... Bp : 회귀 계수

⇒ ML 회귀 예측의 목적 :피처와 결정 값 데이터를 통해 **최적의 회귀 계수**를 찾아내는 것!



*회귀 vs 분류 타겟 개념 혼동 주의!
타겟=정답데이터(y) ⇒ 분류 : 레이블(카테고리) // 회귀 : 결정값(연속적 분포)

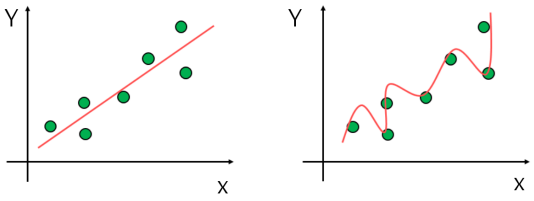
지도 학습의 종류 : 분류//회귀



회귀의 종류 :

독립변수 개수	회귀 계수의 결합
1개 : 단일 회귀	선형: 선형회귀*
여러개 : 다중 회귀	~선형 : 비선형 회귀

*선형 vs 비선형



선형 회귀 : 여러 회귀 중 가장 많이 사용. 실제 값과 예측 값의 차이(오류(Residual)^2)를 최소화하는 직선형 회귀선을 최적화 하는 방식. ⇒ 가장 잘 맞는 직선 찾기

규제(Regularization)에 따라 별도의 유형으로 나뉠 수 있음.

- 일반 선형 회귀 : 예측값과 실제 값의 RSS(Residual Sum of Squares)를 최소화할 수 있도록 회귀 계수를 최적화함. 규제X
- 릿지(Ridge) : 선형 회귀 + L2규제

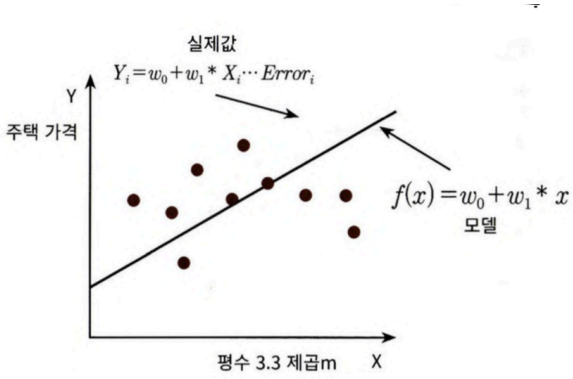
    - L2규제 : 상대적으로 큰 회귀 계수의 예측 영향도를 감소시키기 위해 회귀 계수값을 더 작게 만드는 규제 모델 → 회귀 계수값과 예측 영향도는 양의 상관관계니까
- 라쏘(Lasso) : 선형회귀 + L1 규제
    - L1규제=피처 선택 기능 : 예측 영향력이 작은(=회귀 계수 작은)피처의 회귀 계수를 0으로 만들어 회귀 예측 시 피처가 선택되지 않게
- 엘라스틱넷(ElasticNet) : L1 + L2규제, 피처가 많은 데이터 세트에 적용됨.
- 로지스틱 회귀(Logistic Regression) : 분류에 사용되는 선형 모델. 근데 회귀라는 이름이 붙은... 매우 강력한 분류 알고리즘. 이진 분류 뿐 아니라 희소 영역(ex. 텍스트 분류)에서 뛰어난 예측 성능 보임.

5-2. {단순 선형 회귀}를 통한 회귀 이해

단순 선형 회귀 :

독립변수 하나& 종속 변수 하나 ~ 단일 회귀에 포함됨!

<주택크기-주택가격 그래프>





예측값 f(x) ↔ 실제값 Yi #개념 혼동 주의! 서로 다른 값임.

{단순 선형 회귀}에서의 f(x) 구조 : 1차 함수로 모델링 : 기울기 w1, 절편(intercept) w0으로 구성.

실젯값 = 예측값(1차 함수 값)+ 오류값(residual, 잔차)

⇒최적의 회귀 모델 = 전체 데이터의 잔차 합이 최소가 되는 모델 = **최적의 회귀 계수** 모델

RSS 방식 :

회귀의 비용 함수(Cost function)

$$RSS(w_0, w_1) = \frac{1}{N}\sum_{i=1}^{N}(y_i - (w_0 + w_1 * x_i))^2$$

(i는 1부터 학습 데이터의 총 건수 N까지)

w(회귀계수) 변수가 중심 변수이며 이때 독립, 종속 변수는 모두 상수로 간주

RSS는 비용(cost)이며 w 변수로 구성되는 RSS는 비용함수라고 함.

비용 함수가 반환하는 값 = 오류값

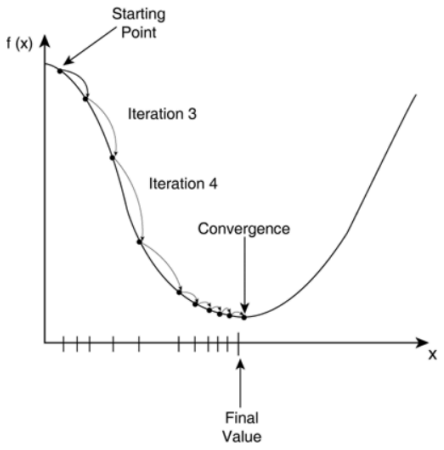
⇒ 오류값을 지속해서 감소시키다 더 이상 감소하지 않는 최소 오류 값을 구해야!

5-3. 비용 최소하기-경사하강법(Gradient Descent)소개

고차원 방정식<경사 하강법

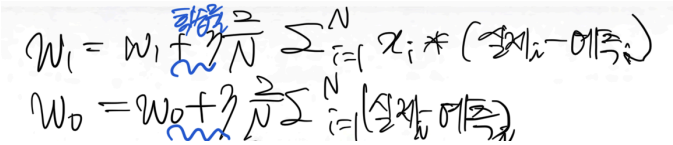
경사 하강법 :

데이터를 기반으로 알고리즘이 스스로 학습한다(머신러닝의 concept)를 가능케 만든 핵심 기법 중 하나



반복적으로 비용 함수의 반환 값(=예측값)과 실제 값의 차이가 작아지는 방향성을 갖고 W파라미터를 지속적으로 보정해 나감.

경사 하강법 수식으로 이해하기-편미분을 통한 비용함수의 최솟값:



*보정계수 = 학습률, 고정된 상수임.

*N = 학습 데이터의 총 건수!!

이렇게 비용 함수가 감소하는 방향성으로 주어진 횟수만큼 w1, w0을 갱신

경사하강법 코딩하기:

1. 임의의 데이터 세트 제작

⇒

```
#임의의 데이터 세트 제작 코드 일부

#y=4x+6을 근사 (w1=4, w0=6). 임의의 값은 노이즈를 위해 만듦
X=2*np.random.rand(100, 1)
y=6+4*X+np.random.randn(100,1)
```

2. 비용 함수 정의 ⇒ get_cost(y, y_pred)

```
def get_cost(y, y_pred):
    N=len(y) #학습데이터 총 건수
    cost = np.sum(np.square(y-y_pred))/N
    return cost
```

3. 함수 정의해 경사하강법 구현

⇒ gradiant_descent()

w1, w0초기화 하기

iters 개수만큼 반복하며 갱신

무작위로 생성한 X,y를 입력 받는데 얘네 모두 넘파이*

3-1. 입력 배열 X값에 대한 예측 배열 y_pred는 np.dot(X, w1.T) + w0으로 구함. ⇒ get_weight_updtates(w1, w0, X, y, learning_rate=0.01)

- 100개의 데이터 X(1~100)이 있다면

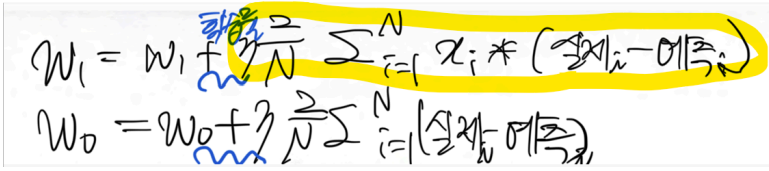
$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{100} \end{bmatrix} = 6 + 4 \times \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_{100} \end{bmatrix} + \begin{bmatrix} \text{noise}_1 \\ \text{noise}_2 \\ \vdots \\ \text{noise}_{100} \end{bmatrix}$$

이때 데이터 X(1~100)과 w1의 계산은

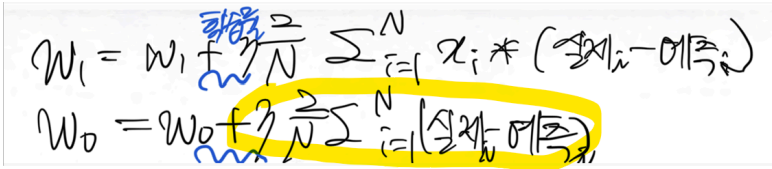
> 💡 입력 배열 X와 w1배열의 내적과 동일

⇒ 넘파이의 내적 계산인 dot()를 이용해서 y_pred=np.dot(X, w1.T) + w0로 예측 배열 값 계산

- w1_update =



- w2_update =



⇒ w1_update랑 w2_update를 넘파이의 dot() 행렬 연산으로 계산한 뒤 이를 반환

```
def get_weight_updates(w1, w0, X, y, learning_rate=0.01):
    N = len(y)
    #먼저 w1,w0_update를 각각 w1, w0의 shape와 동일한 크기를 가진
0값으로 초기화
    w1_update = np.zeros_like(w1)
    w0_update = np.zeros_like(w0)
    #예측배열 계산하고 예측과 실제 값의 차이 계산
    y_pred = np.dot(X, w1.T) + w0
    diff = y-y_pred

    #w0_update를 dot 행렬 연산으로 구하기 위해 모두 1값을 가진 행렬
생성
    #왜? w0은 X데이터 바뀌어도 그냥 의무적으로 내적하되
    #내적으로 인해선 값이 변하면 안되므로
    w0_factors = np.ones((N,1))

    #w1, w0 업데이트를 할 w1,w0_update 계산
    w1_update = -(2/N)*learning_rate*(np.dot(X.T, diff))
    w0_update = -(2/N)*learning_rate*(np.dot(w0_factors.T, di
ff))

    return w1_update, w0_update
```

3-2. get_weight_updates()을 경사 하강 방식으로 반복적으로 수행하여 w1, w0 업데이트하는 함수 gradient_descend_steps() 생성

```
def gradient_descent_steps(X, y, iters=10000):
    #w1,0 모두 초기화
```

```
    w0=np.zeros((1,1))
    w1=np.zeros((1,1))

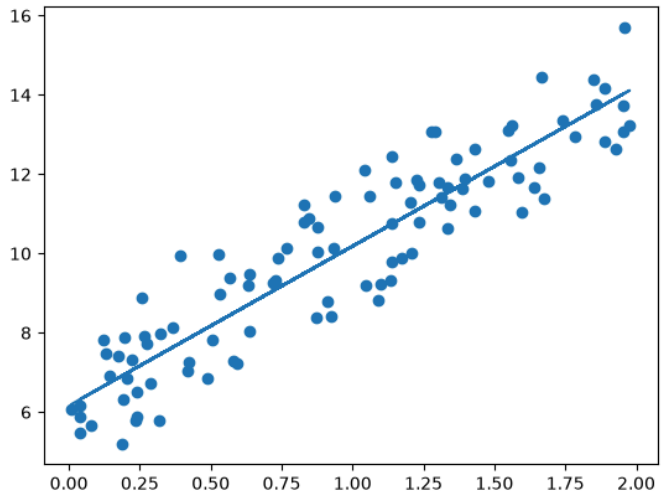
#인자로 주어진 iters만큼 반복적으로 get_weight~함수 호출해서 w1, w0
갱신
for ind in range(iters):
    w1_update, w0_update = get_weight_updates(w1, w0, X, y, l
earning_rate=0.01)
    w1 = w1 - w1_update
    w0 = w0 -w0_update

return w1, w0
```

4. 최종적으로 수행-예측 오류 비용 함수 생성해 결과 확인

```
def get_cost(y, y_pred):
    N = len(y)
    cost = np.sum(np.square(y-y_pred))/N
    return cost

w1, w0 = gradient_descent_steps(X, y, iters=1000)
print("w1:{0:.3f} w0:{1:.3f}".format(w1[0, 0], w0[0, 0]))
y_pred = w1[0, 0]*X + w0
print('Gradient Descent Total Cost : {0:.4f}'.format(get_cost
(y, y_pred)))
```



실전에선 경사하강법은 모든 학습 데이터에 대해 반복적으로 비용 함수 최소화를 위한 값을 업데이트함 → 수행 시간 오래 걸림

⇒ (미니 배치)**확률적 경사 하강법(Stochastic Gradient Descent)** 이용

확률적 경사 하강법 :

전체 데이터가 아닌 일부 데이터만 이용해 w 갱신 ⇒ 경사 하강법에 비해 빠른 속도 보장.

```
def stochastic_gradient_descent_steps(X, y, batch_size=10, iters=1000):
    w0 = np.zeros((1, 1))
    w1 = np.zeros((1,1))
    for ind in range(iters):
        np.random.seed(ind)
        #전체 X, y 데이터에서 랜덤하게 batch_size만큼 데이터를 추출해 Sample_X, sample_y로 저장
        stochastic_random_index = np.random.permutation(X.shape[0])
        sample_X = X[stochastic_random_index[0:batch_size]]
        sample_y = y[stochastic_random_index[0:batch_size]]
```

```
        w1_update, w0_update = get_weight_updates(w1, w0, sam
ple_X, sample_y, learning_rate=0.01)
        w1 = w1 - w1_update
        w0 = w0 - w0_update

    return w1, w0
```

⇒ 기존 경사 하강법으로 구한 예측 오류비용보다 조금 높게 나오지만, 큰 예측 성능 상의 차이는 없으므로 큰 데이터를 처리할 경우엔 걍 확률적 경사 하강법 ㄱㄱ

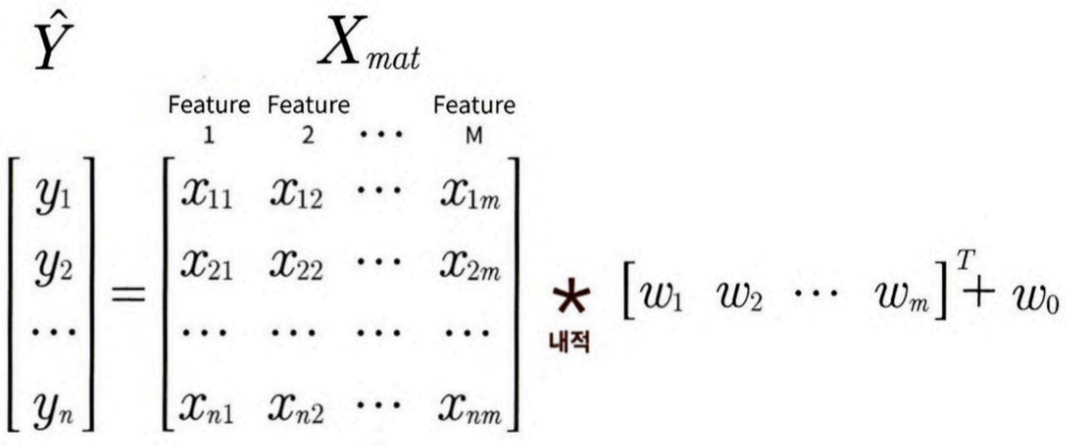
피처가 여러 개인 경우의 회귀 계수 도출 :

피처가 여러 개일 때 예측 회귀식 :

f(w) = w0 + w1*X1 + w2*X2 + w3*X3 + w4*X4 +...+w100*X100

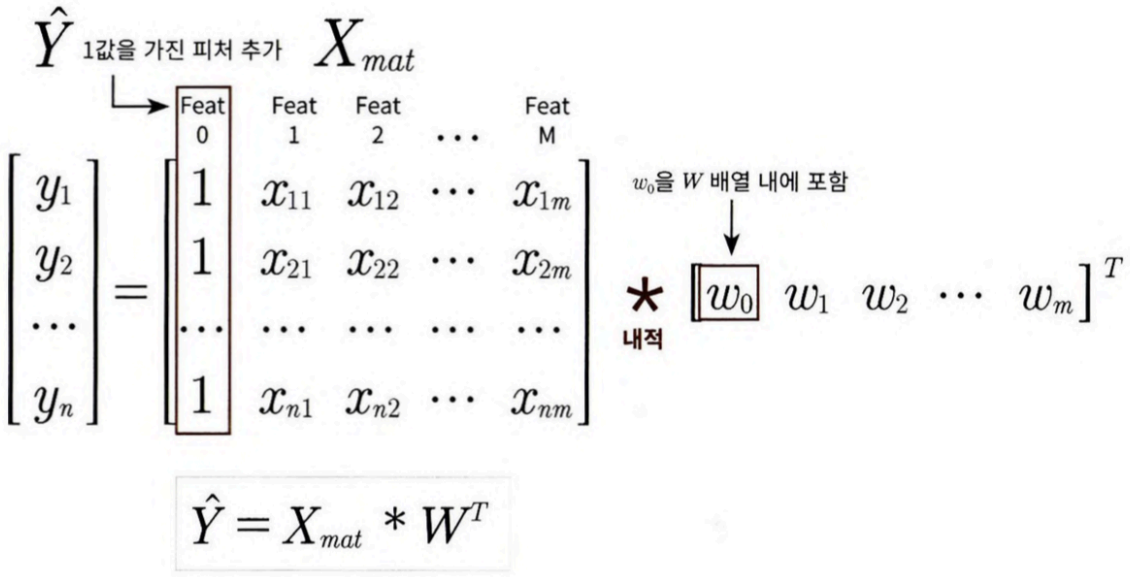
앞선 예제) 입력 행렬X에 대해서 예측 행렬y_pred는 굳이 X의 개별 원소(=데이터)와 w1을 곱해서 구하지 않고,
np.dot(X, w1.T) +w0 #내적계산
을 이용해 계산

이를 응용하면...



*데이터 개수 n개, 피처 m개

w0도 행렬에 넣어주면...



5-4. 사이킷런 LinearRegression을 이용한 보스턴 주택 가격 예측

LinearRegression 클래스-Ordinary Least Squares :

예측값과 실제값의 RSS를 최소화하는 OLS*(Ordinary Least Squares) 추정 방식으로 구현한 클래스

fit()메서드로 X,y 배열을 입력받으면 회귀계수(Coefficients)인 W를 coef_ 속성에 저장함.

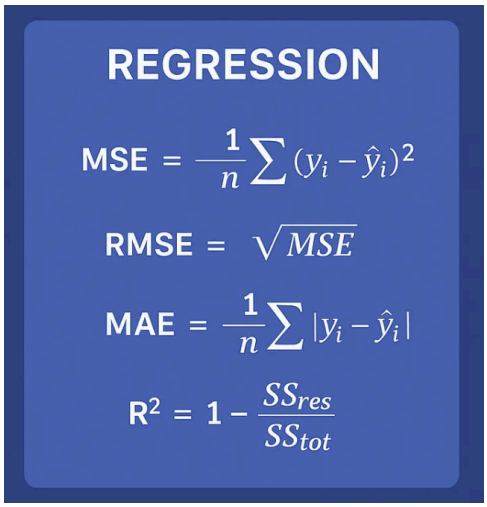
```
#클래스 정의(API specificatioin) 구문*
*class 키워드는 새로운 클래스를 만들 때만 쓰는 예약어
class sklearn.linear_model.LinearRegression(fit_intercept=True, normalize=False, copy_X=True, n_jobs=1)
```

파라미터 : fit_intercept : 절편 계산 여부, normalize : 회귀 수행 전 입력 데이터 세트 정규화

OLS 기반의 회귀 계산은 입력 피처의 독립성이 낮을 경우(=피처 간 상관관계가 높은 경우) 분산이 커져서 오류에 민감해짐 ⇒ 중요한 피처만 남기고 제거하거나 규제를 적용. or PCA를 통해 차원 축소를 수행하는 것도 고려할 수 있음.*

회귀 평가 지표 :

실제 값과 회귀 예측값의 차이 값을 기반으로 한 지표가 중심



사이킷런의 API 및 cross_val_score 나 GridSearchCV에서 평가에 사용되는 scoring 파라미터로 회귀 평가 지표를 구현.

**RMSE는 따로 제공되지 않아 MSE에 제곱근을 씌워 계산하는 식을 별도로 만들어야 함.

cross_val_score, GridSearchCV와 같은 scoring 함수에 회귀 평가 지표를 적용할 때 한 가지 유의 사항 :

사이킷런의 Scoring 함수는 score값이 클수록 좋은 평가 결과로 자동 평가함 , MAE는 절대값의 합이기에 음수가 될 수 없음 ⇒ 실제 값과 예측값의 오류 차이가 큰 양수라면, 실제론 나쁜 모델이지만, Scoring함수는 큰 양수 값을 갖으므로 좋게 평가함. ⇒ MAE에 -1을 곱해 작은 오류값이 더 큰 숫자로 인식하게 해야

1. 데이터 세트를 로드하고 DataFrame으로 변경

bostonDF.info() → 데이터 세트 피처의 Null 값은 없으며 float 형임

2. 각 칼럼이 회귀 결과에 미치는 영향 시각화

seaborn의 replot()함수 → X,Y 축 산점도, 선형회귀 직선

matplotlib.subplots()함수 → 각 ax 마다 칼럼과 PRICE 관계

인자로 입력되는 ncols는 열 방향으로 위치할 그래프 개수, nrows는 행 방향으로 위치할 그래프 개수

⇒그래프 생성 결과 RM, LSTAT의 PRICE영향도가 가장 두드러짐.

RM : 방개수, LSTAT : 하위계층비율

3. LinearRegression 클래스를 이용해 보스턴 주택 가격의 회귀 모델 만들기

train_test_split()→학습//테스트 데이터 세트 분리

metrics모듈의 mean_squared_error(), r2_score() → MSE, R2 Score

LinearRegression으로 생성한 주택 가격 모델의 intercept, coefficients 값 확인

판다스 Series의 sort_values() → coef_ 속성은 회귀 계수 값만 가지고 있으므로 이를 피처별 회귀 계수 값으로 다시 매핑하고 높은 순으로 출력

```
RM          3.4
CHAS        3.0
RAD         0.4
ZN          0.1
INDUS       0.0
B           0.0
TAX        -0.0
AGE         0.0
CRIM       -0.1
LSTAT      -0.6
PTRATIO    -0.9
DIS        -1.7
NOX       -19.8
dtype: float64
```

⇒ RM이 양의 값으로 회귀계수, NOX 피처의 음의 회귀계수 값이 매우 큼. 최적화를 수행하며 coefficients의 변화도 살피기

5개의 폴드 세트에서 cross_val_score() → 교차 검증으로 MSE,RMSE 측정

5-5. 다항 회귀와 과(대)적합/과소적합 이해

다항 회귀 :

Polynomial Regression

$$Y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_p x^p + \epsilon$$

...is still linear regression

💡 다항 회귀는 선형 회귀. 회귀에서 선형/비선형 회귀를 나누는 기준은 회귀 계수가 선형/비선형인지에 따른 것이지 독립 변수의 선형/비선형 여부와 무관.
데이터 세트에 대한 피처 X에 대해 Target Y 값의 관계를 그래프로 표현하면 곡선 형태지만 선형 회귀긴 함.

선형 회귀 때문에 비선형 함수를 선형 모델에 적용시키는 방법을 사용해 구현

PolynomialFeatures 클래스를 통해 degree 파라미터(degree=2)를 통해 입력 받은 단항식 피처를 (2차) 다항식 피처로 변환하기

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures

# 원본 데이터 X 생성 (2x2 행렬)
X = np.arange(4).reshape(2, 2) #0~3까지를 2x2행렬로
print("일차 단항식 계수 피처 : \n", X)

# 2차 다항식 피처 변환기 생성 및 변환
poly = PolynomialFeatures(degree=2)
poly.fit(X)
poly_ftr = poly.transform(X)
print("변환된 2차 다항식 계수 피처:\n", poly_ftr)
```

출력결과)

```
일차 단항식 계수 피처 :
 [[0 1]
 [2 3]]
변환된 2차 다항식 계수 피처:
 [[1. 0. 1. 0. 0. 1.]
 [1. 2. 3. 4. 6. 9.]]
```

출력 결과 해석 :

단항식 계수 피처 : 원본 데이터의 개별 피처인 x1, x2 자체를 말함. 기본 1차 피처(1차 항).

첫 번째 행 : [0,1] 두 번째 행 : [2,3]

첫 번째 행:

첫번째 피처 x1 = 0, 두번째 피처 x2 = 1

PolynomialFeatures는 피처 x1, x2가 주어졌을 때 2차 다항식 형태 조합을 만들어 새로운 피처 열들을 생성함.

PolynomialFeatures(degree=2)로 생성되는 피처항 조합 순서 :

$$\text{변환 형태} = [1, x_1, x_2, x_1^2, x_1 \cdot x_2, x_2^2]$$

1: 편향(Bias) 항 (0차)

x_1: 1차 피처 (원본 피처 1)

x_2: 1차 피처 (원본 피처 2)

x_1^2: x_1의 제곱항 (2차)

$x_1 \cdot x_2$: 두 피처의 상호작용항 (2차)

x_2^2: x_2의 제곱항 (2차)

앞서 첫번째 행의 x1=0, x2=1의 값을 대입하면

$$
\begin{aligned}
&1 \rightarrow \mathbf{1} \\
&x_1 \rightarrow \mathbf{0} \\
&x_2 \rightarrow \mathbf{1} \\
&x_1^2 = 0^2 \rightarrow \mathbf{0} \\
&x_1 \cdot x_2 = 0 \times 1 \rightarrow \mathbf{0} \\
&x_2^2 = 1^2 \rightarrow \mathbf{1}
\end{aligned}
$$

결과 : [1. 0. 1. 0. 0. 1.]

두번째 행의 x1=2, x2=3의 값을 대입하면

$$
\begin{aligned}
&1 \rightarrow \mathbf{1} \\
&x_1 \rightarrow \mathbf{2} \\
&x_2 \rightarrow \mathbf{3} \\
&x_1^2 = 2^2 \rightarrow \mathbf{4} \\
&x_1 \cdot x_2 = 2 \times 3 \rightarrow \mathbf{6} \\
&x_2^2 = 3^2 \rightarrow \mathbf{9}
\end{aligned}
$$

결과 : [1. 2. 3. 4. 6. 9.]

이렇게 변환된 Polynomial 피처에 선형 회귀를 적용해 다항 회귀를 구현

3차 다항 계수로 3차 다항 회귀 함수식을 유도해보기

-PolynomialFeatures로 피처를 변환한 후 LinearRegression 클래스로 다항 회귀를 구현

: 정답 공식을 우리가 먼저 정해놓고 데이터를 생성한 뒤, 머신런이 모델에게 그 데이터를 주었을 때 원래 우리가 설정했던 정답공식의 계수들을 다시 찾아내는지 테스트

step1. 3차 다항 회귀 함수를 임의로 설정 후 회귀 계수를 예측하기

- 3차 다항 회귀 결정 함수식(정답 공식)을 설정, 이를 위한 함수 polynomial_func() 제작. 이 함수는 3차 다항 계수 피처 값(x의 값)이 입력되면 결정 값(y값)을 반환함.

```python
def polynomial_func(X): #정답 공식
    y = 1 + 2*X[:,0] + 3*X[:,0]**2 + 4*X[:,1]**3
    return y

X = np.arange(4).reshape(2,2) #원본 데이터 피처
print('일차 단항식 계수 feature : \n',X)
y = polynomial_func(X)
print('삼차 다항식 결정값 : \n', y) #
```

✓ 0.0s Python

```
일차 단항식 계수 feature :
 [[0 1]
 [2 3]]
삼차 다항식 결정값 :
 [  5 125]
```

- X : 일차 단항식 계수 feature, y : 삼차 다항식 결정값

step2. 1차 단항식 계수를 삼차 다항식 계수로 변환, 이를 선형 회귀에 적용해 다항 회귀로 구현

- 일차 단항식 계수를 3차 다항식 계수로 변환 (위에 일차→2차 변환과 동일 과정)
- 이를 선형 회귀에 적용

```python
poly_ftr = PolynomialFeatures(degree=3).fit_transform(X)
print('3차 다항식 계수 feature : \n', poly_ftr)

model = LinearRegression()
model.fit(poly_ftr, y)
print('Polynomial 회귀 계수 \n', np.round(model.coef_, 2))
print('Polynomail 회귀 Shape : ', model.coef_.shape)
```

✓ 0.0s Python

```
3차 다항식 계수 feature :
 [[ 1.  0.  1.  0.  0.  1.  0.  0.  0.  1.]
 [ 1.  2.  3.  4.  6.  9.  8. 12. 18. 27.]]
Polynomial 회귀 계수
 [-0.    0.16  0.16  0.32  0.49  0.65  0.65  0.97  1.46  2.11]
Polynomail 회귀 Shape :  (10,)
```

- 1차 단항식 계수 피처는 2개(x1, x2) 였지만, PolynomialFeatures(degree=3)로 만든 3차 다항식 계수 피처는 10개(상수항~3차항)로 늘어남.

  ⇒ 직선(선형)을 곡선(비선형)으로 표현할 수 있게 됨을 의미

- LinearRegression을 통해 3차 다항 회귀를 적용하면 회귀 계수가 10개로 늘어남. 그 결과는 출력 결과의 Polynomial 회귀 계수 값으로 알 수 있음.

- 원래의 정답인 [1, 2, 0, 3,0, 0, 0, 0, 0, 4]와는 차이가 있지만 다항 회귀로 근사하고 있음을 알 수 있음.

> 💡 계수 피처와 회귀 계수 혼동 주의! :
> 계수 피처 = 독립변수 X = Feature
> : 예측을 수행하기 위해 모델에 입력으로 들어가는 데이터의 특성
> 타깃(y)값에 영향을 미치는 원인
> 회귀 계수 = w
> :각 피처가 타깃값에 얼마나 중요한 영향을 미치는지 계산하는 핵심 파라미터.
> 머신러닝 모델이 학습(fit)하는 과정에서 비용함수를 최소화하도록 스스로 찾아내는 값

⇒ PolynomialFeatures로 피처를 변환한 후 LinearRegression 클래스로 다항 회귀를 구현 = 두 단계를 걸침

⇒두 단계 걸치는 것보다 ...

-Pipeline 객체를 이용해 한번에 다항회귀 구현

```
model = Pipeline([('poly', PolynomialFeatures(degree=3)),
                                        ('linear', LinearRegressi
on())])
print('Polynomial 회귀 계수\n', np.round(model.named_steps['li
near'].coef_, 2))
```

출력 결과)

Polynomial 회귀 계수
[-0. 0.16 0.16 0.32 0.49 0.65 0.65 0.97 1.46 2.11]

다항 회귀를 이용한 과소적합 및 과적합 이해

다항 회귀 차수 → 과적합 문제 발생 정도

다항 회귀를 이용한 과소적합 및 과적합 이해 예제

피처X → target y : 노이즈가 포함된 다항식의 코사인 그래프 관계

1. 학습 데이터 생성 - 30개의 임의의 데이터 X와 X의 코사인 값에 약간의 잡음 변동 값을 더한 target인 y
2. 예측 결과를 비교할 다항식 차수를 각각 [1,4 15]로 리스트화해서 예측 결과 비교

- 다항식 차수 별로 학습을 수행한 뒤 cross_val_score()로 MSE 값을 구해 차수별 예측 성능 평가
- 0~1까지 균일하게 구성된 100개의 테스트용 데이터 세트를 이용해 차수별 회귀 예측 곡선 그리기
- 예측 결과 비교)

  실선으로 표현된 예측 곡선 = 다항 회귀 예측 곡선

  점선으로 표현된 곡선 = 실제 데이터 세트 X,Y의 코사인 곡선

  Degree 1 : 과소적합, MSE = 약0.41

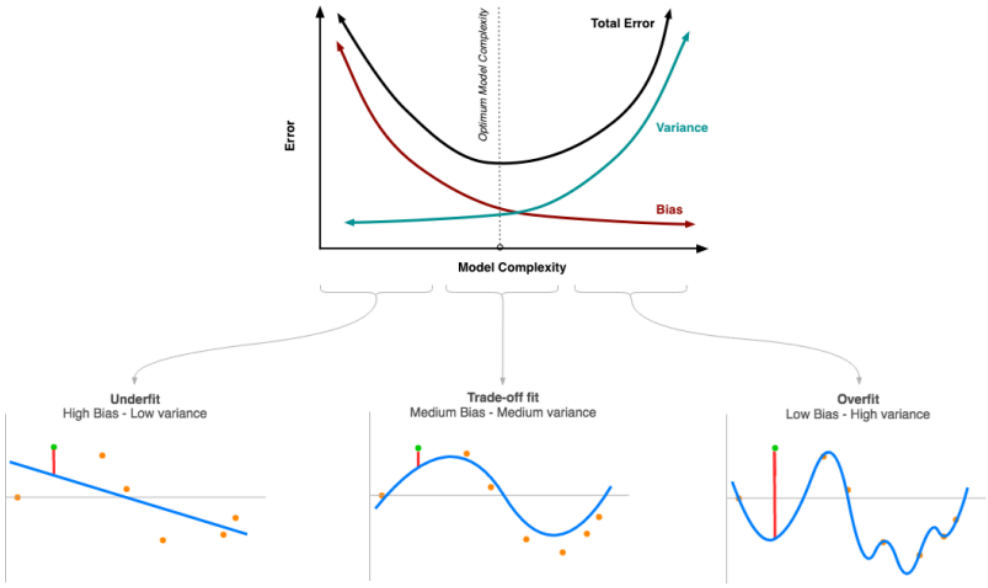
  Degree 4 : 실제 데이터 세트와 유사, MSE = 약 0.04

  Degree 15 : 과적합, MSE = 엄청 크게 나오는 오류 발생. 예측 곡선을 보면 예측 곡선이 학습 데이터 세트만 정확히 예측하고, 테스트 값의 실제 곡선과는 완전히 다른 형태의 예측 곡선이 만들어짐.

편향 - 분산 트레이드 오프 :

단순화된 모델 → 고편향성 → 근접 정도 낮음

복잡한 모델 → 고분산성 → 밀집 정도 낮음



골디락스 지점 : 편향을 낮추고 분산을 높이면서 전체 오류가 가장 낮아지는 지점

높은편향/낮은분산 → 과소적합 위험

낮은편향/높은분산 → 과적합 위험

5-6. 규제 선형 모델 - 릿지, 라쏘, 엘라스틱넷

최적의 모델을 위한 비용함수 = RSS & 회귀 계수 크기 제어

→ RSS최소화 방법과 회귀 계수 값을 제어하는 균형을 맞춰야 함.

$$\text{비용함수목표} = Min(RSS(W) + alpha * //W//$$

alpha : 학습 데이터 적합 정도와 회귀 계수의 값 크기를 제어하는 튜닝 파라미터

//W// : W벡터, 회귀 계수

alpha값 크게 → 회귀 계수 W값 작게해야

alpha값 작게 → RSS 최소화를 중점적으로 고려해야

⇒ 규제 : alpha의 값을 지속적으로 증가시키면 회귀 계수(//W//)값의 크기를 감소시킬 수 있음
→ 과적합 개선

L1 Regularization

$$\text{Cost} = \sum_{i=0}^{N}(y_i - \sum_{j=0}^{M} x_{ij}W_j)^2 + \lambda \sum_{j=0}^{M} |W_j|$$

L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^{N}(y_i - \sum_{j=0}^{M} x_{ij}W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^{M} W_j^2}_{\text{Regularization Term}}$$

L1 규제 : W의 절댓값에 대해 페널티 부여 ⇒ 라쏘 회귀

L2 규제 : W의 제곱에 대해 페널티 부여 ⇒ 릿지 회귀

릿지 회귀

사이킷런은 Ridge 클래스를 통해 릿지 회귀 구현

alpha : Ridge 클래스의 주요 생성 파라미터. 이는 L2 규제 계수에 해당.

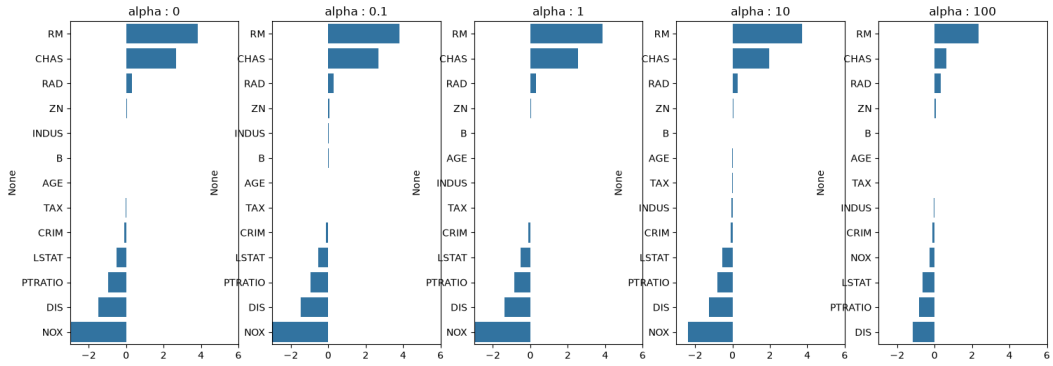
릿지 회귀 실습 :

- 알파 값을 늘려가며 RMSE와 회귀 계수 값의 변화 살펴보기

```
alpha 0 일 때 5 folds의 평균 RMSE : 5.829
alpha 0.1 일 때 5 folds의 평균 RMSE : 5.788
alpha 1 일 때 5 folds의 평균 RMSE : 5.653
alpha 10 일 때 5 folds의 평균 RMSE : 5.518
alpha 100 일 때 5 folds의 평균 RMSE : 5.330
```

⇒ alpha가 100일 때 평균 RMSE가 5.330으로 가장 좋음.

- 알파 값 변화에 따른 피처의 회귀 계수 값을 가로 막대 그래프로 시각화



	alpha : 0	alpha : 0.1	alpha : 1	alpha : 10	alpha : 100
RM	3.809865	3.818233	3.854000	3.702272	2.334536
CHAS	2.686734	2.670019	2.552393	1.952021	0.638335
RAD	0.306049	0.303515	0.290142	0.279596	0.315358
ZN	0.046420	0.046572	0.047443	0.049579	0.054496
INDUS	0.020559	0.015999	-0.008805	-0.042962	-0.052826
B	0.009312	0.009368	0.009673	0.010037	0.009393
AGE	0.000692	-0.000269	-0.005415	-0.010707	0.001212
TAX	-0.012335	-0.012421	-0.012912	-0.013993	-0.015856
CRIM	-0.108011	-0.107474	-0.104595	-0.101435	-0.102202
LSTAT	-0.524758	-0.525966	-0.533343	-0.559366	-0.660764
PTRATIO	-0.952747	-0.940759	-0.876074	-0.797945	-0.829218
DIS	-1.475567	-1.459626	-1.372654	-1.248808	-1.153390
NOX	-17.766611	-16.684645	-10.777015	-2.371619	-0.262847

⇒alpha 값이 증가함에 따라 회귀 계수 값은 작아짐. 특히 NOX 피처의 경우 그 변화가 두드러짐.

라쏘 회귀

- L1규제를 선형 회귀에 적용한 것. L2규제가 회귀 계수의 크기를 감소시키는 데 반해(L2는 회귀 계수 0으로 만들진 않음), L1 규제는 불필요한 회귀 계수를 급격히 감소시켜 0으로 만들어 제거. ⇒ 적절한 피처만 회귀에 포함 시킴.
- 사이킷런은 Lasso 클래스를 통해 라쏘 회귀 구현
- alpha : Ridge 클래스의 주요 생성 파라미터. 이는 L1 규제 계수에 해당.

라쏘 회귀 실습 :

- 알파 값을 변화시키며 RMSE와 각 피처의 회귀 계수 출력.
- 엘라스틱넷도 알파값 변화시키며 결과 출력

  get_linear_reg_eval()함수 생성 : 인자로 회귀모델이름, alpha 값들 리스트, 피처데이터 세트, 타깃데이터 세트 입력 → alpha값에 따른 폴드 평균 RMSE출력 후 회귀 계수 값들을 DataFrame으로 반환

	alpha : 0.07	alpha : 0.1	alpha : 0.5	alpha : 1	alpha : 3
RM	3.789725	3.703202	2.498212	0.949811	0.000000
CHAS	1.434343	0.955190	0.000000	0.000000	0.000000
RAD	0.270936	0.274707	0.277451	0.264206	0.061864
ZN	0.049059	0.049211	0.049544	0.049165	0.037231
B	0.010248	0.010249	0.009469	0.008247	0.006510
NOX	-0.000000	-0.000000	-0.000000	-0.000000	0.000000
AGE	-0.011706	-0.010037	0.003604	0.020910	0.042495
TAX	-0.014290	-0.014570	-0.015442	-0.015212	-0.008602
INDUS	-0.042120	-0.036619	-0.005253	-0.000000	-0.000000
CRIM	-0.098193	-0.097894	-0.083289	-0.063437	-0.000000
LSTAT	-0.560431	-0.568769	-0.656290	-0.761115	-0.807679
PTRATIO	-0.765107	-0.770654	-0.758752	-0.722966	-0.265072
DIS	-1.176583	-1.160538	-0.936605	-0.668790	-0.000000

⇒ alpha 크기 값이 증가하며 일부 피처의 회귀 계수는 아예 0으로 바뀜.

엘라스틱넷 회귀

L1과 L2 규제를 결합한 회귀.

RSS(W) + alpha2*//W//^2 + alpha1*//W//^1 최소화하는 W찾기

라쏘 회귀(L1규제)가 서로 상관관계가 높은 피처들의 경우에 이들 중에서 중요한 피처만 셀렉션하고 다른 피처들은 모두 회귀 계수를 0으로 만드는 성향 강함

⇒ alpha 값에 따라 회귀 계수의 값이 급격히 변동할 수 있음 ⇒ L2규제를 추가해 이 경향성 완화

사이킷런은 ElasticNet 클래스를 통해 릿지 회귀 구현

ElasticNet 클래스의 주요 생성 파라미터:

alpha : a*L1 + b*L2로 정의됨

l1_ratio : a/a+b

*a : L1규제의 alpha 값 / b : L2규제의 alpha 값

엘라스틱넷 회귀 실습 :

- alpha 값 변화시키며 (l1_ratio는 고정)RMSE와 각 피처의 회귀 계수들 출력

	alpha : 0.07	alpha : 0.1	alpha : 0.5	alpha : 1	alpha : 3
RM	3.574162	3.414154	1.918419	0.938789	0.000000
CHAS	1.330724	0.979706	0.000000	0.000000	0.000000
RAD	0.278880	0.283443	0.300761	0.289299	0.146846
ZN	0.050107	0.050617	0.052878	0.052136	0.038268
B	0.010122	0.010067	0.009114	0.008320	0.007020
AGE	-0.010116	-0.008276	0.007760	0.020348	0.043446
TAX	-0.014522	-0.014814	-0.016046	-0.016218	-0.011417
INDUS	-0.044855	-0.042719	-0.023252	-0.000000	-0.000000
CRIM	-0.099468	-0.099213	-0.089070	-0.073577	-0.019058
NOX	-0.175072	-0.000000	-0.000000	-0.000000	-0.000000
LSTAT	-0.574822	-0.587702	-0.693861	-0.760457	-0.800368
PTRATIO	-0.779498	-0.784725	-0.790969	-0.738672	-0.423065
DIS	-1.189438	-1.173647	-0.975902	-0.725174	-0.031208

⇒alpha 값에 따른 피처들의 회귀 계수 값이 라쏘보다는 상대적으로 0이 되는 값이 적음을 알 수 있음. (L2 규제에 의해)

선형 회귀 모델을 위한 데이터 변환

선형 모델의 특징 :

피처-타깃 간 선형의 관계가 있다고 가정. 이 최적의 선형 함수를 찾아내 결괏값을 예측해야!

선형 회귀 모델은 피처값과 타깃값의 분포가 정규 분포 형태인 걸 선호.

⇒선형 회귀 모델을 적용하기 전 먼저 데이터에 대한 스케일링/정규화 작업을 수행해야!

피처 데이터 세트와 타깃 데이터 세트에 스케일링/정규화 작업을 수행하는 방법은 조금 다름 :

피처 데이터 세트의 경우 :

1. StandardScaler클래스로 평균 0, 분산 1인 표준 정규 분포를 가진 데이터 세트로 변환
2. MinMaxScaler클래스로 최솟값 0, 최댓값 1인 값으로 정규화
3. 스케일링/정규화를 수행한 데이터 세트에 "다시" 다항 특성을 적용해 변환(다항변환)
4. 로그 변환 → 정규 분포에 가까운 형태로 값이 분포됨. 가장 많이 쓰이는 방식

데이터 변환 실습 :

- 변환을 차례로 적용하며 RMSE로 각 경우별 예측 성능을 측정하기 위한 get_scaled_data() 함수 생성 : method 인자로 변환 방법 결정 → 변환 선택
- Ridge 클래스의 alpha 값을 변화시키며 피처 데이터 세트를 여러 가지 방법으로 변환한 데이터 세트 입력해 RMSE 변화 확인

⇒ 변환된 피처 데이터 :

(None, None) : 아무런 변환 X 원본 데이터

('Standard', None) : 표준 정규 분포

('Standard', 2) : 표준 정규 분포를 다시 2차 다항식으로 변환

('MinMax', None) : 최솟값 최댓값 정규화

('MinMax', 2) : 표준 정규 분포를 다시 2차 다항식으로 변환

('Log', None) : 로그 변환

⇒ 표준 정규 분포로 일차 변환 + 2차 다항식 변환 → 성능 개선

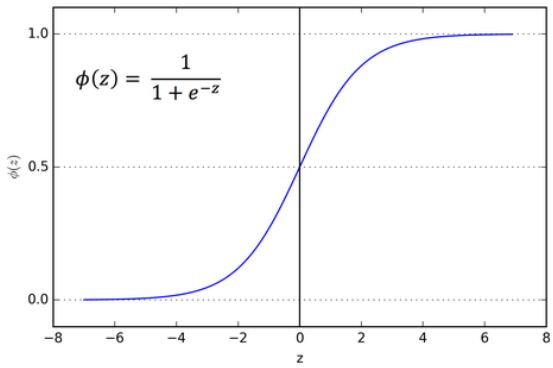
최솟값/최댓값 정규화로 일차 변환 + 2차 다항식 변환 → 성능 개선

로그변환 → 대부분 alpha 값에서 좋은 성능 향상

⇒ 선형 회귀를 적용하려는 데이터 세트에 데이터 값의 분포가 심하게 왜곡되어 있을 경우에 이처럼 로그 변환을 적용하는 것이 좋음

5-7. 로지스틱 회귀

로지스틱 회귀는 분류에 사용되지만 선형 회귀 계열



학습을 통해 시그모이드 함수 최적선을 찾고 시그모이드 함수의 반환 값 = 확률로 간주해 확률에 따라 분류를 결정.

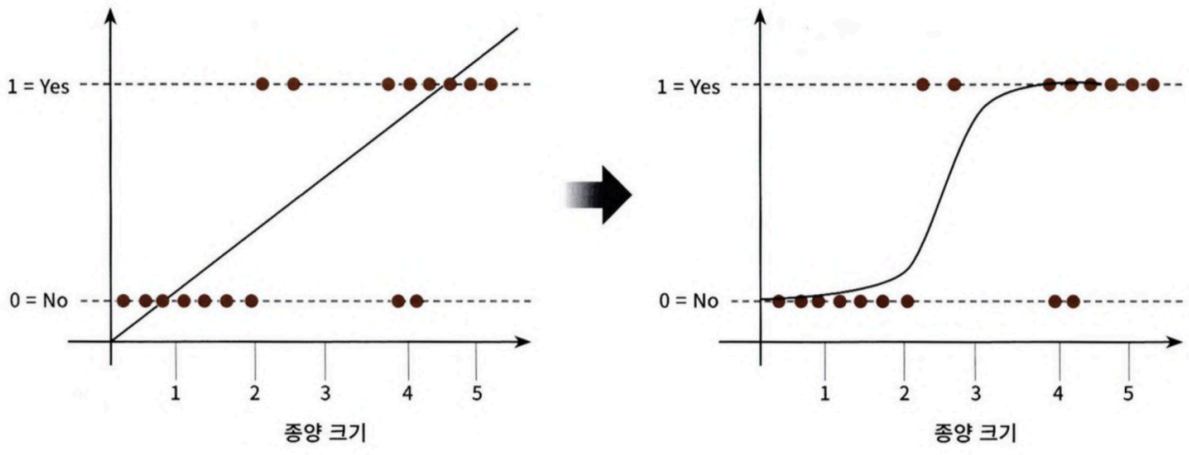
↔ 일반 선형회귀 : 학습을 통해 선형 함수의 최적선을 찾음

시그모이드 함수 (고정적) 특징 : 0과 1 사이, z=가운데(?) 축일 땐 0.5

로지스틱 회귀를 이용한 분류 문제 - 연속적인 값이 아닌 레이블 값 예측

1과 0의 값으로 악성 종양(1) // ~악성 종양(0) 예측

선형 회귀vs로지스틱회귀 (0 or 1이므로 0과 1사이 값을 갖을 수 없음)



사이킷런 : LogisticRegression 클래스 제공

LogisticRegression 클래스에서 solver 파라미터의 lbfgs, liblinear, newton-cg, sag, saga 값을 적용해 최적화 선택 가능*

⇒ 대부분 lbfgs 또는 liblinear를 선택함

LogisticRegression 클래스를 이용한 위스콘신 유방암 데이터 세트로 로지스틱 회귀를 이용해 암 여부 판단

1. 사이킷런의 load_breat_cancer() 호출해 전체 데이터 세트 생성
2. 로지스틱 회귀→선형회귀계열 → 정규 분포도에 따라 예측 성능 영향을 있음 ⇒ 정규 분포 형태의 표준 스케일링을 적용한 후 train_test_split()을 이용해 데이터 세트 분리
3. 로지스틱 회귀를 이용해 학습 및 예측 수행, 정확도와 ROC-AUC 값 구하기

   solver 값을 'lgfbs'로 설정할 경우 성능 : accuracy : 0.977, roc_auc : 0.995
4. 서로 다른 solver 값으로 LogisticRegression 학습하고 성능 평가

   :

   solver:lbfgs, accuracy:0.977, roc_auc : 0.995
   solver:liblinear, accuracy:0.982, roc_auc : 0.995
   solver:newton-cg, accuracy:0.977, roc_auc : 0.995
   solver:sag, accuracy:0.982, roc_auc : 0.995
   solver:saga, accuracy:0.982, roc_auc : 0.995

   ⇒ 데이터 세트가 워낙 작아 solver간 성능 결과 차가 크지 않음
5. GridSearchCV ⇒ 위스콘신 데이터 세트에서 solver, penalty, C 최적화

   :

   최적 하이퍼 파라미터 : {'C': 0.1, 'penalty': 'l2', 'solver': 'liblinear'}, 최적 평균 정확도 : 0.979

   +FitFailedWarning 메시지가 함께 나오는데 이는 solver가 lbfgs일 때 L1 규제를 지원하지 않음에도 GridSearchCV가 L1 규제값을 입력했기 때문

LogisticRegression클래스의 주요 하이퍼 파라미터 : Solver, max_iter, penalty, C

penalty : 규제의 유형을 설정 → l1, l2, 기본은 l2

C : 규제의 강도 조절하는 alpha 값의 역수

L1, L2 규제는 solver 설정에 따라 영향을 받음.

- 로지스틱 회귀는 가볍고 빠르면서 이진 분류예측 성능도 뛰어남.
- 이 때문에 로지스틱 회귀를 이진 분류의 기본 모델로 사용하는 경우가 많음.
- 로지스틱 회귀는 희소한 데이터 세트 분류에도 뛰어난 성능을 자주 보여 텍스트 분류에도 자주 사용됨.

5-8. 회귀 트리

: (선형/비선형 같은) 회귀 함수를 기반으로 하지 않고 (회귀)트리를 기반으로 하는 회귀 방식

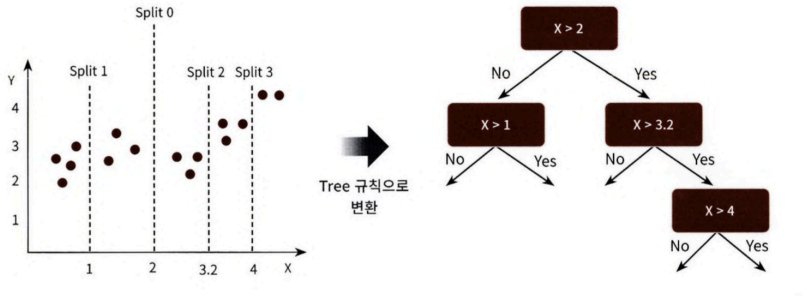
회귀를 위한 트리 생성 → 회귀 예측

회귀트리 : 리프 노드에 속한 데이터 값의 평균값을 구해 회귀 예측값을 계산

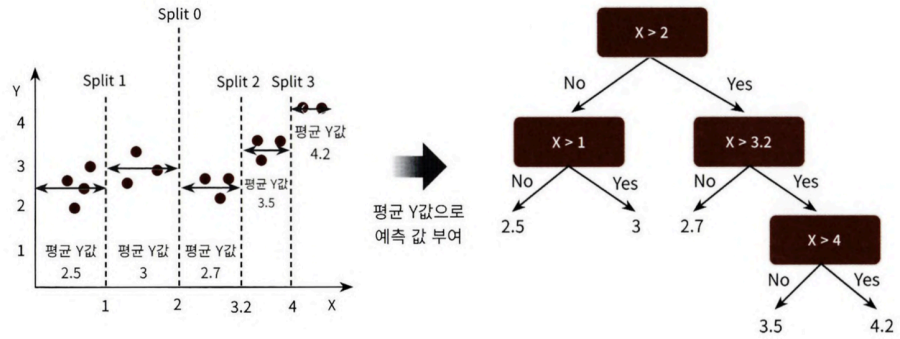
회귀트리 동작 과정 :

X피처 데이터 세트와 결정값 Y가 2차원 평면상에 있다면...

1. X값의 균일도를 반영한 지니계수 → X피처를 결정 트리 기반으로 분할
2. 루프노트를 split 0 기준으로 분할
3. 이렇게 분할된 규칙 노드에서 다시 split1, split2 규칙노드로 분할
4. split2는 다시 재귀적으로 split3 규칙 노드로...



5. 리프 노드 생성 기준에 부합하는 트리 분할이 완료됐다면 리프 노드에 소속된 데이터 값의 평균값을 구해 최종적으로 리프 노드에 결정 값으로 할당



모든 트리 기반 알고리즘(결정 트리, 랜덤 포레스트, GBM, XGBoost, LightGBM)은 CART 알고리즘에 기반하기에 분류 뿐 아니라 회귀도 가능함

트리기반 알고리즘에서 CART 기반 회귀 수행 → 사이킷런의 Estimator클래스

알고리즘	회귀 Esimator 클래스	분류 Estimator 클래스
Decision Tree	DecisionTreeRegressor	DecisionTreeClassifier
Gradient Boosting	GradientBoostingRegressor	GradientBoostingClassifier
XGBoost	XGBRegressor	XGBClassifier
LightGBM	LGBMRegressor	LGBMClassifier

1. 사이킷런의 랜덤포레스트 회귀 트리인 RandomForestRegressor를 이용해 앞의 선형 회귀에서 다룬 보스턴 주택 가격 예측 수행

   :

```
5 교차 검증의 개별 Negative MSE scores:  [ -7.88 -13.15 -20.79 -46.17 -18.7 ]
5 교차 검증의 개별 RMSE scores:  [2.81 3.63 4.56 6.8  4.32]
5 교차 검증의 개별 평균 RMSE scores: 4.422
```

2. 결정트리, GBM, XGBoost, LightGBM의 Regressor 모두 이용해 보스턴 주택 가격 예측 수행

   get_model_cv() 함수 : 입력 모델과 데이터 세트 → 교차 검증으로 평균 RMSE 계산

   :

```
#####  DecisionTreeRegressor  #####
 5 교차 검증의 평균 RMSE : 4.422
#####  RandomForestRegressor  #####
 5 교차 검증의 평균 RMSE : 4.422
#####  GradientBoostingRegressor  #####
 5 교차 검증의 평균 RMSE : 4.422
#####  XGBRegressor  #####
 5 교차 검증의 평균 RMSE : 4.422
#####  LGBMRegressor  #####
 5 교차 검증의 평균 RMSE : 4.422
```

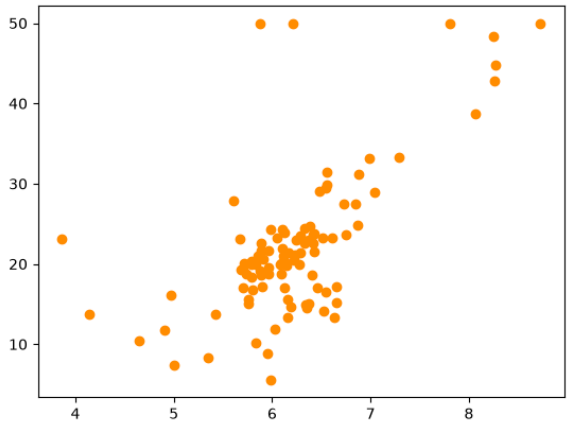
- 회귀 트리 Regressor 클래스는 선형 회귀와 다른 처리 방식이므로 회귀 계수를 제공하는 coef_ 속성이 없음 ⇒ feature_importances_를 이용해 피처의 중요도를 알 수 있음.

3. 회귀 트리 Regressor와 선형 회귀의 예측값 판단 비교(시각화) - PRICE와 가장 밀접한 양의 상관관계를 갖는 RM 칼럼만 이용해 선형 회귀와 결정 트리로 PRICE 예측 회귀선 표현

   데이터 개수의 100개만 샘플링하고 RM, PRICE 칼럼만 추출.

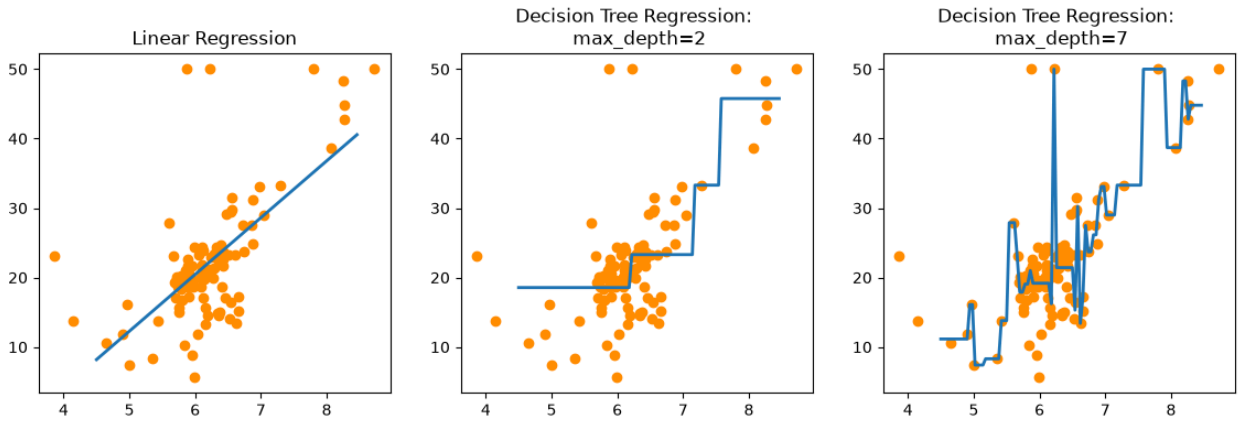
   x축 : 독립변수 RM, y축 : 종속변수 PRICE

   <데이터 세트를 산점도로 표현>



   LinearRegression과 DecisionTreeRegressor를 max_depth를 각각 2, 7로 해서 학습, 이렇게 학습된 Regressor에 RM값을 4.5~8.5까지의 100개 테스트 데이터 세트로 제공했을 때의 예측값을 구하기.

   <예측한 회귀선 표현>



⇒

선형 회귀 : 직선으로 예측 회귀선 표현

회귀 트리 : 분할되는 데이터 지점에 따라 브랜치 만들며 계단 형태로 회귀선 만듦.
DecisionTreeRegressor의 max_depth=7인 경우에 과적합 됨.